

R3.04 : Design patterns

Patrons de conception (3)

Arnaud Lanoix Brauer

`Arnaud.Lanoix@univ-nantes.fr`



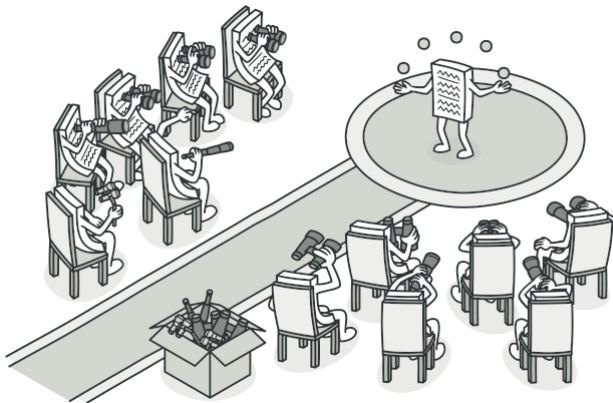
IUT Nantes
Pôle Sciences et technologie

Nantes Université

Département informatique

- 1 Patrons de conception comportementaux
- 2 Patrons de conception structurels
- 3 Conclusion

Patron Observateur – Observer pattern



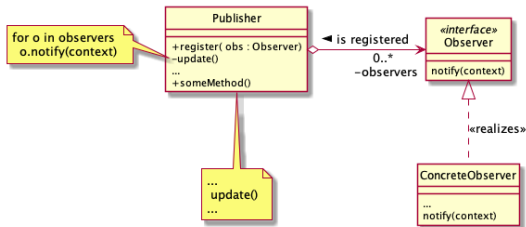
Patron Observateur – Observer pattern

Problème

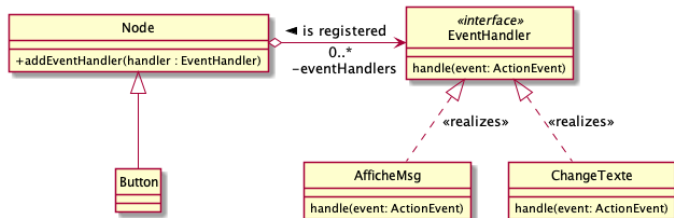
Informer **automatiquement** un ou plusieurs objets (= **les observateurs**) du **changement d'état** d'un objet donné (= **l'observable**),

Solution

- Les observateurs implémentent tous une méthode de **notification** (ie comment ils réagiront au changement ?)
- L'objet observable connaît ses **observateurs**
- A chaque changement d'état, il **notifie** tous ses observateurs (en appelant leurs méthodes de **notification**)



Exemple d'observateurs : les événements JavaFX

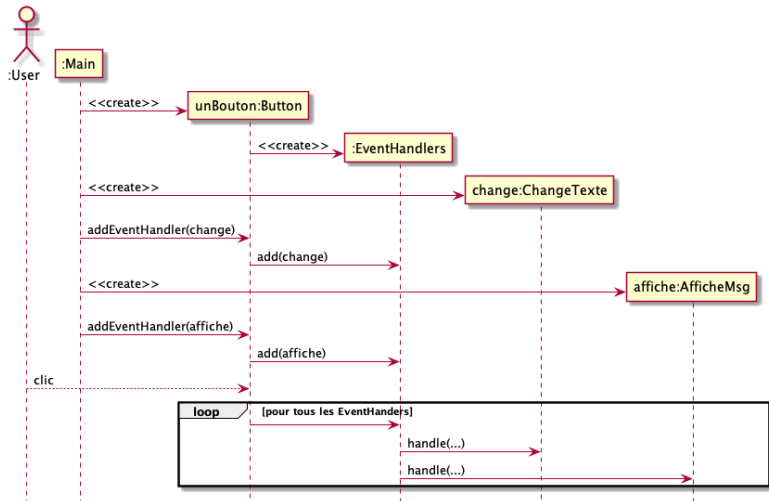


```
...
val unBouton = Button("Go")
val texte = TextField()
unBouton.addEventHandler(ActionEvent.ACTION,
                        ChangeTexte(texte))
unBouton.addEventHandler(ActionEvent.ACTION,
                        AfficheMsg())
...
```

```
class ChangeTexte(val texte : TextField)
    : EventHandler<ActionEvent> {
    override fun handle(event: ActionEvent?) {
        texte.text = "OK"
    }
}
```

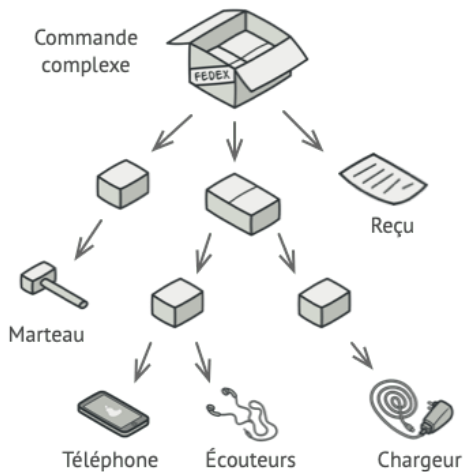
```
class AfficheMsg : EventHandler<ActionEvent> {
    override fun handle(event: ActionEvent?) {
        println("Bouton cliqué")
    }
}
```

Exemple d'observateurs : les événements JavaFX (2)



- 1 Patrons de conception comportementaux
- 2 Patrons de conception structurels
- 3 Conclusion

Patron Composite – Composite pattern



Patron Composite – Composite pattern

Problème

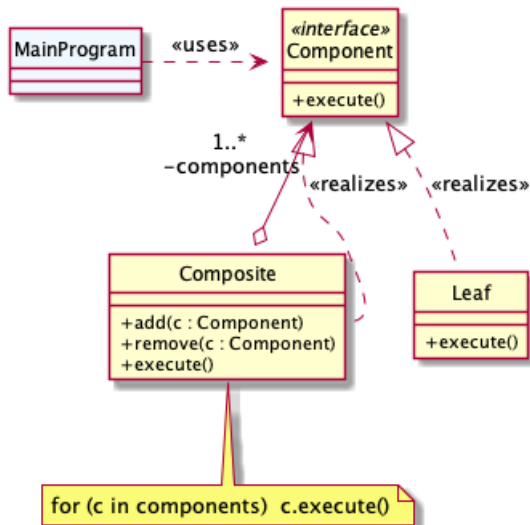
Comment représenter une structure **arborescente** ?

Solution

Les **composants-feuilles** et les **composite-conteneurs** implémentent une **même interface**

- En pratique, l'utilisateur n'aura pas à **distinguer** entre les **objets primitifs** et les **conteneurs**.

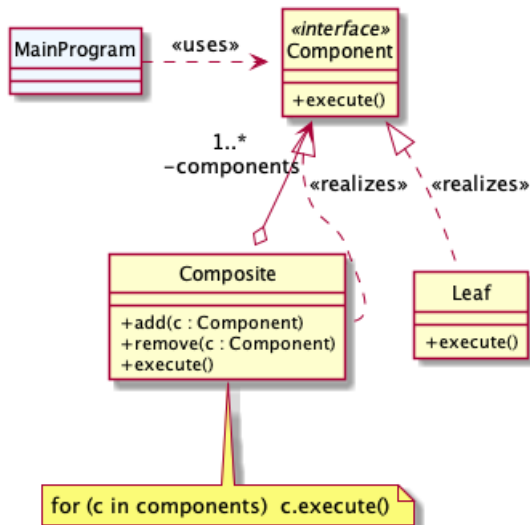
Patron Composite : schéma UML



Notations UML

Notez l'utilisation du "◇" sur l'association, indiquant une **aggrégation** UML "◇" possible, pour indiquer une **composition** UML

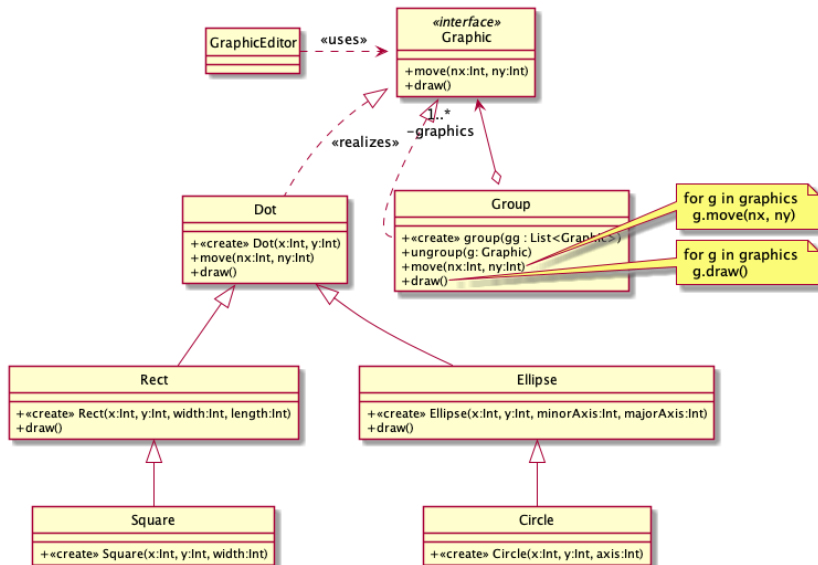
Patron Composite : schéma UML



Notations UML

Notez l'utilisation du "◇" sur l'association, indiquant une **aggrégation** UML "◇" possible, pour indiquer une **composition** UML

Exemple de structure Composite



Patron Délégation – Delegate pattern (=composition)

Problème

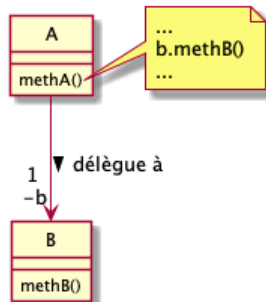
Implémenter dans une classe A un (des) traitement(s) complexe(s) alors qu'on connaît une autre classe B qui sait déjà faire ce(s) traitement(s)

Solution

Déléguer à la classe B les traitements à réaliser, en ajoutant à A un **attribut** de type B et en **appelant** les méthodes de B dans A
⇒ on parle de **composition**

On doit pouvoir remplacer la classe B sans **impacter** l'usage de A

⇒ Delegate souvent combiné avec Strategy



Délégation $\neq$ utilisation

pour qu'il y ait **délégation**, il faut qu'il y ait un attribut

Patron Délégation – Delegate pattern (=composition)

Problème

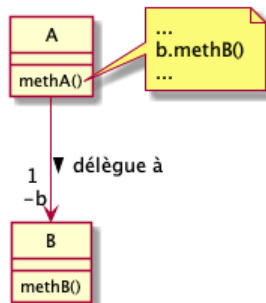
Implémenter dans une classe A un (des) traitement(s) complexe(s) alors qu'on connaît une autre classe B qui sait déjà faire ce(s) traitement(s)

Solution

Déléguer à la classe B les traitements à réaliser, en ajoutant à A un **attribut** de type B et en **appelant** les méthodes de B dans A
⇒ on parle de **composition**

On doit pouvoir remplacer la classe B sans **impacter** l'usage de A

⇒ **Delegate** souvent combiné avec **Strategy**



Délégation $\neq$ utilisation

pour qu'il y ait **délégation**, il faut qu'il y ait un attribut

Patron Délégation – Delegate pattern (=composition)

Problème

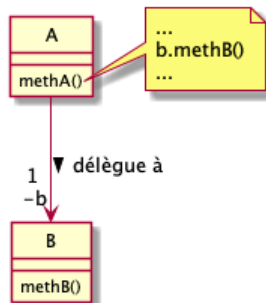
Implémenter dans une classe A un (des) traitement(s) complexe(s) alors qu'on connaît une autre classe B qui sait déjà faire ce(s) traitement(s)

Solution

Déléguer à la classe B les traitements à réaliser, en ajoutant à A un **attribut** de type B et en **appelant** les méthodes de B dans A
⇒ on parle de **composition**

On doit pouvoir remplacer la classe B sans **impacter** l'usage de A

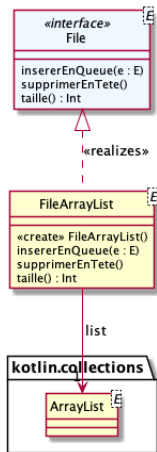
⇒ **Delegate** souvent combiné avec **Strategy**



Délégation $\neq$ utilisation

pour qu'il y ait **délégation**, il faut qu'il y ait un attribut

Exemple de délégation



```
class FileArrayList<E> : File<E> {
    val list = ArrayList<E>()

    override fun insererEnQueue(element: E) {
        list.add(element)
    }

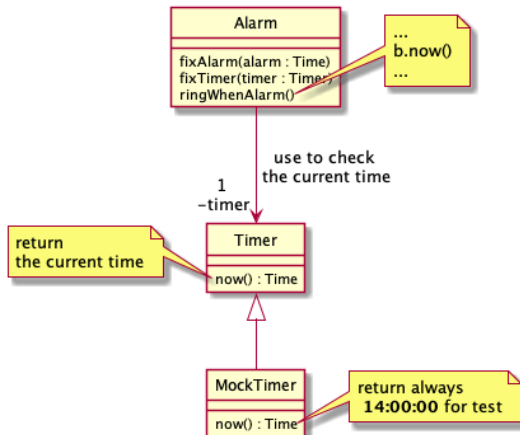
    override fun supprimerEnTete() {
        list.removeFirst()
    }

    override fun taille(): Int {
        return list.size
    }
    ...
}
```

Testabilité et injection de dépendances

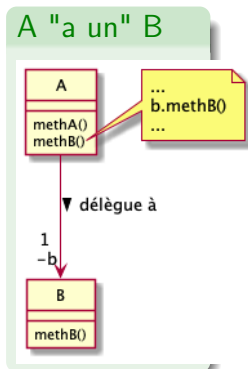
Injection de dépendances

Construire une classe par **composition** en déléguant certains traitements à des classes spécifiques augmente la **testabilité** de la classe en permettant de substituer la classe par un **mock**.

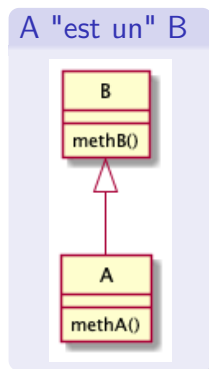


Délégation (directe) vs. héritage

Souvent, on peut hésiter entre **délégation** ou **héritage**



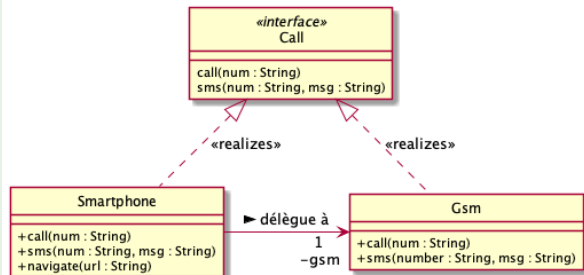
VS.



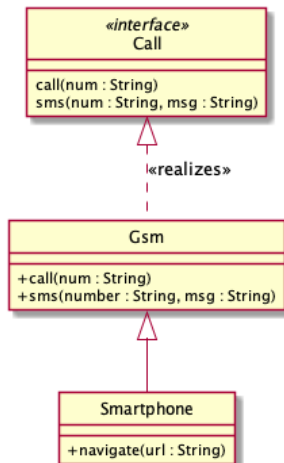
- L'objet "B" peut être réutilisé dans plusieurs classes différentes
- La délégation permet l'**injection de dépendances**

Délégation vs. héritage : exemple

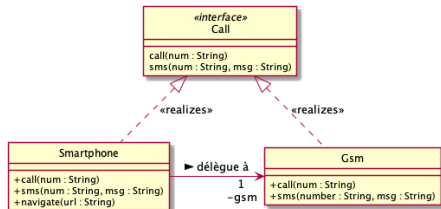
Smartphone "a un" Gsm



Smartphone "est un" Gsm



Délégation directe en Kotlin grâce à `by`



```
class Gsm : Call {
    override fun call(num: String) {
        println("call $num")
    }

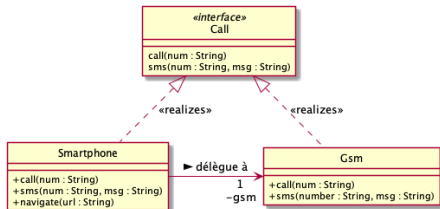
    override fun sms(num: String,
        msg: String) {
        println("send $msg to $num")
    }
}
```

```
class Smartphone(val gsm : Gsm)
    : Call {
    override fun call(num: String) {
        gsm.call(num)
    }
    override fun sms(num: String,
        msg: String) {
        gsm.sms(num, msg)
    }
    fun navigate(url : String) {
        println("go to $url")
    }
}
```

Version simplifiée utilisant `by`

```
class Smartphone(val gsm : Gsm)
    : Call by gsm {
    fun navigate(url : String) {
        println("go to $url")
    }
}
```

Délégation directe en Kotlin grâce à **by**



```
class Gsm : Call {
    override fun call(num: String) {
        println("call $num")
    }

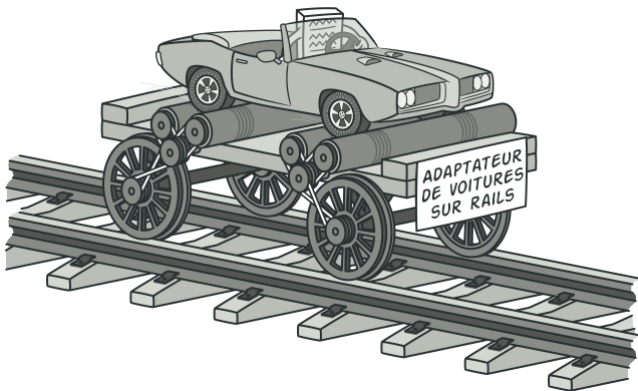
    override fun sms(num: String,
        msg: String) {
        println("send $msg to $num")
    }
}
```

```
class Smartphone(val gsm : Gsm)
    : Call {
    override fun call(num: String) {
        gsm.call(num)
    }
    override fun sms(num: String,
        msg: String) {
        gsm.sms(num, msg)
    }
    fun navigate(url : String) {
        println("go to $url")
    }
}
```

Version simplifiée utilisant **by**

```
class Smartphone(val gsm : Gsm)
    : Call by gsm {
    fun navigate(url : String) {
        println("go to $url")
    }
}
```

Patron Adaptateur – Adapter pattern



Patron Adaptateur – Adapter pattern

Problème

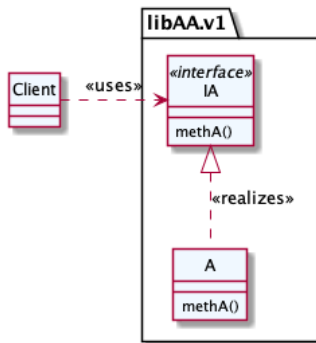
Comment **continuer à utiliser** une bibliothèque dont l'interface a été **modifiée** sans toucher au reste du programme ?

Solution

Une nouvelle classe va **exposer** l'ancienne interface et **utiliser** les méthodes de la nouvelle pour réaliser l'ancienne interface

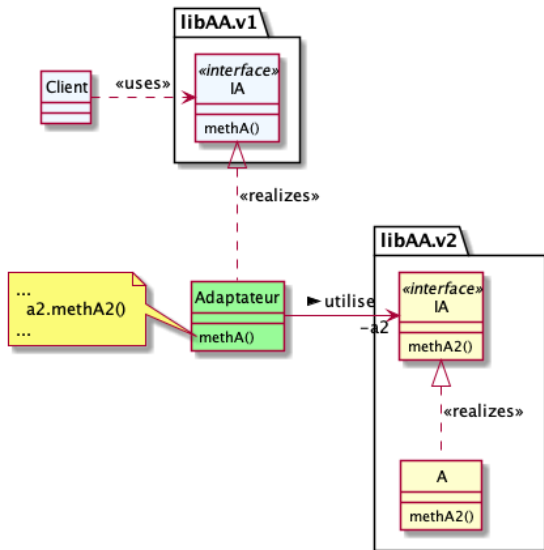
Adaptateur : schéma UML

La librairie libAA est mise à jour : l'interface d'utilisation a changée



Adaptateur : schéma UML

La librairie libAA est **mise à jour** : l'interface d'utilisation a changée



Adapter – précisions

- Adaptateur **converti** l'interface d'une classe en une autre forme **conforme** à l'attente du client.
- Permet d'interconnecter des classes qui sans cela seraient incompatibles.
- Adaptateur peut également être utiliser pour **remplacer** une bibliothèque par une autre
- Le reste du programme **utilisera** l'adaptateur de manière **transparente**.

Adapter + Delagate

Un adaptateur **delègue** à une (ou plusieurs) autre(s) classe(s) la réalisation des traitements demandés

Pour aller plus loin : patron Proxy

Variante d'Adapter avec une **restriction** de l'interface réalisée

Adapter – précisions

- Adaptateur **converti** l'interface d'une classe en une autre forme **conforme** à l'attente du client.
- Permet d'interconnecter des classes qui sans cela seraient incompatibles.
- Adaptateur peut également être utiliser pour **remplacer** une bibliothèque par une autre
- Le reste du programme **utilisera** l'adaptateur de manière **transparente**.

Adapter + Delagate

Un adaptateur **delègue** à une (ou plusieurs) autre(s) classe(s) la réalisation des traitements demandés

Pour aller plus loin : patron Proxy

Variante d'Adapter avec une **restriction** de l'interface réalisée

Adapter – précisions

- Adaptateur **converti** l'interface d'une classe en une autre forme **conforme** à l'attente du client.
- Permet d'interconnecter des classes qui sans cela seraient incompatibles.
- Adaptateur peut également être utiliser pour **remplacer** une bibliothèque par une autre
- Le reste du programme **utilisera** l'adaptateur de manière **transparente**.

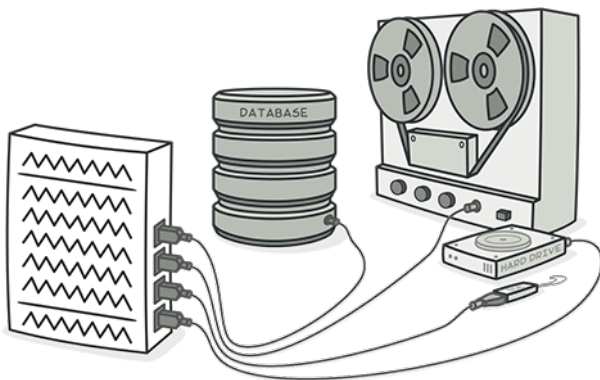
Adapter + Delagate

Un adaptateur **delègue** à une (ou plusieurs) autre(s) classe(s) la réalisation des traitements demandés

Pour aller plus loin : patron Proxy

Variante d'Adapter avec une **restriction** de l'interface réalisée

Patron Entrepôt de données – Repository pattern



Patron Entrepôt de données – Repository pattern

Problème

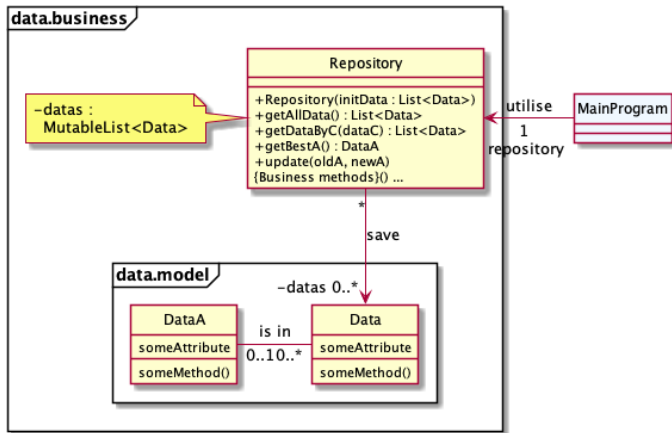
Comment **séparer** les **traitements** des données, des données elles-même ?

Solution

Une classe **Repository** spécifique **stocke** les données et offre toutes les **fonctionnalités métier** nécessaire

- **Repository** a (au moins) un attribut **privé** de type `MutableCollection<E>` pour stocker les données
- **Repository** réalise la **couche "Métier"** de l'application, séparée de la **couche "Données"** : i.e **implémente** les traitements nécessaires

Entrepôt de données : schéma UML



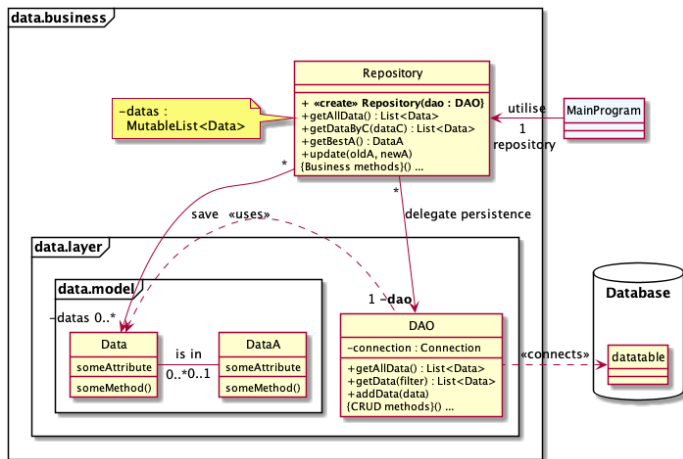
Encapsulation

- L'accès direct aux données est **verrouillé** (attribut **private**)
- Les méthodes renvoient des listes **non-modifiables** : `List<E>`

Entrepôt de données et DAO

DAO vs. Repository

Souvent, **Repository** est "fusionné" avec **DAO** ; il est bien mieux que Repository **délègue** à DAO la persistance des données (**DP Delegate**)



[UML / Kotlin] Attributs dérivés

Attribut dérivé

Il s'agit d'un attribut dont la valeur **se calcule** à partir de la valeur des autres attributs. Il **n'est pas nécessaire de stocker sa valeur**.

Noter "/" en UML

Personne
-nom : String -prenom : String -dateNaissance : LocalDate + / nomComplet : String + / age : Int
+«create» Personne (nom,dateNaissance) ...()

En Kotlin

```
class Personne (  
    private var nom: String,  
    private var prenom : String,  
    private val dateNaissance : LocalDate) {  
  
    val nomComplet : String  
        get() = "$prenom ${nom.uppercase()}"  
    val age: Int  
        get() = LocalDate.now().year  
            - dateNaissance.year  
  
    fun renommer(nouveau : String) {  
        nom = nouveau  
    }  
}
```

[UML / Kotlin] Attributs dérivés

Attribut dérivé

Il s'agit d'un attribut dont la valeur **se calcule** à partir de la valeur des autres attributs. **Il n'est pas nécessaire de stocker sa valeur.**

Noter "/" en UML

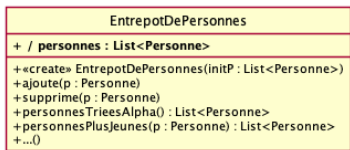
Personne
-nom : String -prenom : String -dateNaissance : LocalDate + / nomComplet : String + / age : Int
+«create» Personne (nom,dateNaissance) ...()

En Kotlin

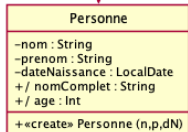
```
class Personne (  
    private var nom: String,  
    private var prenom : String,  
    private val dateNaissance : LocalDate) {  
  
    val nomComplet : String  
        get() = "$prenom ${nom.uppercase()}"  
    val age: Int  
        get() = LocalDate.now().year  
            - dateNaissance.year  
  
    fun renommer(nouveau : String) {  
        nom = nouveau  
    }  
}
```

Entrepôt de données et attribut dérivé

L'attribut **dérivé** `personnes:List<Personne>` donne un accès **en lecture seule** à `_personnes:MutableList<Personne>`



*
↓ stocke
-_personnes 0..*

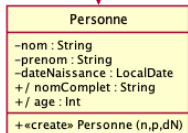
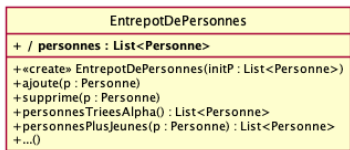


En Kotlin

```
class EntrepotDePersonnes(  
    initP : List<Personne>) {  
  
    private val _personnes : MutableList<Personne>  
        = initP.toMutableList()  
  
    val personnes : List<Personne>  
        get() = _personnes  
  
    fun personnesTrieesAlpha() : List<Personne> {  
        return _personnes.sortedBy  
            { it -> it.nomComple }  
    }  
  
    fun personnesPlusJeunes(p : Personne) : List<Personne> {  
        return _personnes.filter  
            { it -> it.age <= p.age }  
    }  
}
```

Entrepôt de données et attribut dérivé

L'attribut **dérivé** `personnes:List<Personne>` donne un accès **en lecture seule** à `_personnes:MutableList<Personne>`



En Kotlin

```
class EntrepotDePersonnes(  
    initP : List<Personne>) {  
  
    private val _personnes : MutableList<Personne>  
        = initP.toMutableList()  
  
    val personnes : List<Personne>  
        get() = _personnes  
  
    fun personnesTrieesAlpha() : List<Personne> {  
        return _personnes.sortedBy  
            { it -> it.nomComple }  
    }  
  
    fun personnesPlusJeunes(p : Personne) : List<Personne> {  
        return _personnes.filter  
            { it -> it.age <= p.age }  
    }  
}
```

- 1 Patrons de conception comportementaux
- 2 Patrons de conception structurels
- 3 Conclusion

- Fabrique
 - ▶ Fabrique abstraite
 - ▶ Monteur
 - Singleton
 - DAO
 - ▶ Repository
 - Patron de méthode
 - Stratégie
 - Iterateur
 - ▶ Visiteur
 - Observer
 - Composite
 - Délégation
- Adaptateur
 - ▶ Procuration/Proxy
 - ▶ Médiateur
 - Interprête
 - Prototype
 - Pont
 - Décorateur
 - Façade
 - Poids-mouche
 - Chaine de responsabilité
 - Commande
 - Memento
 - Etat
 - ...

Autres Design patterns

- Fabrique
 - ▶ Fabrique abstraite
 - ▶ Monteur
- Singleton
- DAO
 - ▶ Repository
- Patron de méthode
- Stratégie
- Iterateur
 - ▶ Visiteur
- Observer
- Composite
- Délégation
- Adaptateur
 - ▶ Procuration/Proxy
 - ▶ Médiateur
- Interprête
- Prototype
- Pont
- Décorateur
- Façade
- Poids-mouche
- Chaine de responsabilité
- Commande
- Memento
- Etat
- ...

Design-patterns et (éco-)conception

Etude comparative complète sur <https://medium.com> ¹

	Modularité	Evolutivité	Réutilisabilité	Testabilité	Ressources
Factory					
Abstract Factory					
Builder					
Singleton					
DAO					
Repository					
Template method					
Strategy					
Iterator					
Observer					
Composite					
Delegate/Compose					
Adapter					

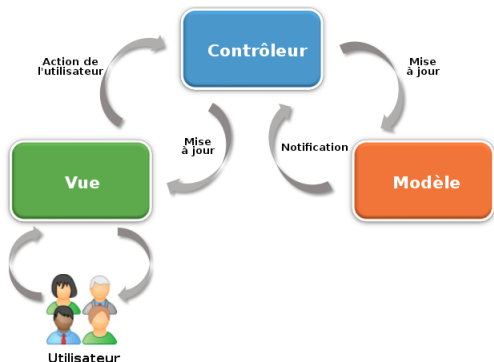
- **Modularité** – est-ce qu'on découple les éléments de la solution ?
- **Evolutivité** – est-ce qu'on maîtrise les changements ?
- **Réutilisabilité** – est-ce qu'on fafiorise la réutilisation ?
- **Testabilité** – est-ce qu'on renforce la capacité à être testé ?
- **Ressources CPU + Mémoire + I/O** – est-ce qu'on minimise la consommation de ressources ?

1. Lien valide le 07/10/2025

Patron d'architecture Modèle-Vue-Contrôleur

Le patron MVC est un **patron architectural** pour les applications graphiques proposant de séparer les "préoccupations" :

- 1 **Modèle** = données à afficher + règles métier
- 2 **Vue** = interface graphique de présentation des données
- 3 **Contrôleur** = logique concernant les actions des utilisateurs ; **modifie** le modèle et la vue



- MVC combine les patrons de conception **Observateur**, **Stratégie** et **Composite** (+ DAO, + Repository, + Fabrique)
- Variantes : **modèle-vue-vue modèle** (MVVM), **modèle-vue-présentation** (MVP)